

• Dynamic Programming

DP is an algorithm technique used to solve problem by :

- 1) Breaking them into smaller subproblems
- 2) Solving each subproblem once
- 3) Storing results to avoid recomputation

• Fibonacci Problem

The fibonacci sequence is defined as

$$F(n) = F(n-1) + F(n-2)$$

with base cases :

$$F(0) = 0, F(1) = 1$$

• Algorithm :

~~Step 1:~~ Input Step 1: Input n

Step 2: Create array $F[0 \dots n]$

Step 3: $F[0] = 0$

Step 4: $F[1] = 1$

Step 5: For $i = 2$ to n

$$F[i] = F[i-1] + F[i-2]$$

Step 6: Return $F[n]$

• Time complexity

Method

Time Complexity

Recursion

$$O(2^n)$$

DP

$$O(n)$$

Space complexity = $O(n)$

If optimized $\rightarrow O(1)$ space

Coin Change Problem (DP) :

Given coin denomination and target amount, find minimum number of coins required to make that amount

1) Coins = {1, 5, 6, 9} w = 10

coins \ w	0	1	2	3	4	5	6	7	8	9	10
1	0	1	2	3	4	5	6	7	8	9	10
5	0	1	2	3	4	1	2	3	4	5	2
6	0	1	2	3	4	1	1	2	3	4	2
9	0	1	2	3	4	1	1	2	3	1	2

require two coins making up the solution :

final ans : $a[n-1][n-1] \rightarrow$ check whether previous ans is similar to previous value

2

check by comparing row backwards

if not same take the coin i.e 5 {5}

Subtract from 10 i.e $10 - 5 = 5$

Now find for column 5 in the 2nd row we get 1

check again

5

1

not same, take the coin i.e

{5, 5}

Subtract 5 from amount i.e $5 - 5 = 0$

Now check for 0 column in second row we get 0

check again

0

0

same, we reach at initial step stop we got two coins

2) Coins {4, 3, 1, 5}
Amount = 7

Amt \ Coins	0	1	2	3	4	5	6	7
4	0	∞	∞	∞	1	∞	∞	∞
3	0	∞	∞	1	1	∞	2	2
1	0	1	2	1	1	2	2	2
5	0	1	2	1	1	1	2	2

Pick 3 $\rightarrow 7-3=4$ Amt ≤ 4 {3}
Pick 4 $\rightarrow 4-4=0$ Amt = 0 {3, 4}

3) Coins {6, 4, 3, 2}
Amount = 8

Amt \ Coins	0	1	2	3	4	5	6	7	8
6	∞	∞	∞	∞	∞	∞	1	∞	∞
4	∞	∞	∞	∞	1	∞	1	∞	2
3	∞	∞	∞	1	1	∞	1	2	2
2	∞	∞	1	1	1	2	1	2	2

Pick 4 Amt = $8-4=4$ {4}
Pick 4 Amt = $4-4=0$ {4, 4}

Algorithm :

Step 1: Input m (number of coins)

Step 2: Input coins $[1..m]$

Step 3: Input amount A

Step 4: Create table $C[0..m][0..A]$

Step 5: For $i = 0$ to m

$C[i][0] = 0$

Step 6: For $j = 1$ to A

$C[0][j] = \infty$

Step 7: For $i = 1$ to m

For $j = 1$ to A

If coins $[i] \leq j$

$C[i][j] = \min(C[i-1][j], 1 + C[i][j - \text{coins}[i]])$

Else $C[i][j] = C[i-1][j]$

$C[i][j] = C[i-1][j]$

Step 8: Result = $C[m][A]$

Time complexity and Space Complexity

$O(m \times n)$

n : amount, m : no. of coins

Recurrence Relation

$C[i][j] = \min(C[i-1][j], 1 + C[i][j - \text{coins}[i]])$

if coins $[i] \leq j$

$= C[i-1][j]$ if coins $[i] > j$

Explanation:

$1 + C[i][j - \text{coins}[i]] \rightarrow$ include current coin

$C[i-1][j] \rightarrow$ exclude current coin

• 0/1 Knapsack Problem

We have n items, each has weight (w_i) and profit (p_i)

Each item can be taken (1) or Not taken (0)

So items cannot be divided which was done in fractional knapsack.

1) weights = {3, 4, 6, 5} $W = 8$

Profits = {2, 3, 1, 4} $n = 4$

		$w \rightarrow$								
	$i \downarrow$	0	1	2	3	4	5	6	7	8
P_i	w_i	0	0	0	0	0	0	0	0	0
2	3	0	0	0	2	2	2	2	2	2
3	4	2	0	0	2	3	3	3	5	5
4	5	3	0	0	2	3	4	4	5	6
1	6	4	0	0	2	3	4	4	5	6

$x_i = \{1, 0, 0, 1\}$

Above cell same don't select current row

Now select second last row

Profit = 6 selected $w_i = 5$ and $p_i = 4$

Profit = $6 - 4 = 2$

Check for 2 in upper row, keep checking till above cell is not same.

Now select first row

Profit = 2 selected $w_i = 3$ and $p_i = 2$

Profit = $2 - 2 = 0$

we get {3, 5} $3 + 5 = 8$

Stop

You can select weights to select $w_i = 5$

Total weight $8 - 5 = 3$

Now find column of 3 repeat till cells are not same, if same select weight 0

$w_i = 3$, Total weight $3 - 3 = 0$

Hence $\{5, 3\}$

2) At $W = 5, n = 4$

weights = $\{2, 1, 3, 2\}$

Profit = $\{12, 10, 20, 15\}$

P_i	w_i	0	1	2	3	4	5
0	0	0	0	0	0	0	0
12	2	1	0	12	12	12	12
10	1	2	0	10	12	22	22
20	3	3	0	10	12	22	30
15	2	4	0	10	12	25	37

Total profit = 37

We take $w_i = 2$

take $w_i = 1$ $7w = 3 - 1 = 2$

take $w_i = 2$ $7w = 2 - 2 = 0$

Item chosen = $\{1, 0, 1, 1\}$ i.e. w_1, w_2 and w_4

Algorithm

Input profit, weights, capacity w

Step 1: Create 2d array $k[0..n][0..w]$

Step 2: For $i=0$ to n

$k[i][0] = 0$

Step 3: For $w=0$ to w

$k[0][w] = 0$

Step 4: For $i=1$ to n

For $w=1$ to w

If $w[i] \leq w$

$k[i][w] = \max(k[i-1][w], p[i] +$

else

$k[i][w] = k[i-1][w]$

Step 5: Maximum profit = $k[n][w]$

Recurrence relation:

If item can fit:

$k[i][w] = \max(k[i-1][w], p[i] + k[i-1][w-w[i]])$

If item cannot:

$k[i][w] = k[i-1][w]$

Time and Space Complexity

$O(n \times w)$

- Floyd - Warshall Algorithm (All pairs Shortest Path)
Used to find shortest path between every pair of vertices in weighted graph.
Works for :

Directed or undirected graphs

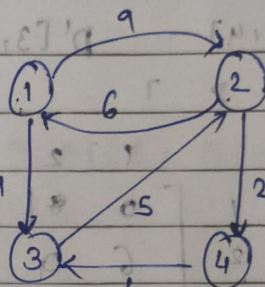
Negative edge weights allowed

But no negative cycle

The algorithm progressively checks and improves shortest path estimates by allowing intermediate vertices one by one.

1)

	1	2	3	4
1	0	9	-4	∞
2	6	0	2	2
3	∞	5	0	∞
4	∞	∞	1	0



$$D^0[2,3] \quad D^0[2,1] + D^0[1,3] \quad D^0 = 1$$

	1	2	3	4
1	0	9	-4	∞
2	6	0	∞	2
3	∞	5	0	∞
4	∞	∞	1	0

$$\infty > 6 + (-4) = +2$$

$$D^0[2,4] \quad D^0[2,1] + D^0[1,4]$$

$$2 < 6 + \infty = \infty$$

$$D^0[3,2] \quad D^0[3,1] + D^0[1,2]$$

$$5 < \infty$$

$$D^0[3,4] \quad D^0[3,1] + D^0[1,4]$$

$$\infty = \infty$$

$$D^0[4,2] \quad D^0[4,1] + D^0[1,2]$$

$$\infty = \infty$$

$$D^0[4,3] \quad D^0[4,1] + D^0[1,3]$$

$$1 < \infty$$

$$D_2 = \begin{bmatrix} 0 & 9 & 11 & 4 \\ 6 & 0 & 2 & 2 \\ 11 & 5 & 0 & 7 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$

$$D^*[1,3] = D^*[1,2] + D^*[2,3]$$

$$-4 < 11$$

$$D^*[1,4] = D^*[1,2] + D^*[2,4]$$

$$D^*[3,1] = D^*[3,2] + D^*[2,1]$$

$$\infty > 11$$

$$D^*[3,4] = D^*[3,2] + D^*[2,4]$$

$$\infty > 7$$

$$D_3 = \begin{bmatrix} 0 & 9 & -4 & 3 \\ 6 & 0 & 2 & 2 \\ 11 & 5 & 0 & 7 \\ 12 & 6 & 1 & 0 \end{bmatrix} \quad \text{By solving}$$

$$D^4 = \begin{bmatrix} 0 & 9 & -4 & 3 \\ 6 & 0 & 2 & 2 \\ 11 & 5 & 0 & 7 \\ 12 & 6 & 1 & 0 \end{bmatrix}$$

This is the final shortest path matrix

• Algorithm (2222) Floydwarshall

(W, n) Floydwarshall

Step 1: Input number of vertices n

Step 2: Create matrix $D[n][n]$

Step 3: Initialize D with graph edge weights

Step 4: For $k=1$ to n

For $i=1$ to n

For $j=1$ to n

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

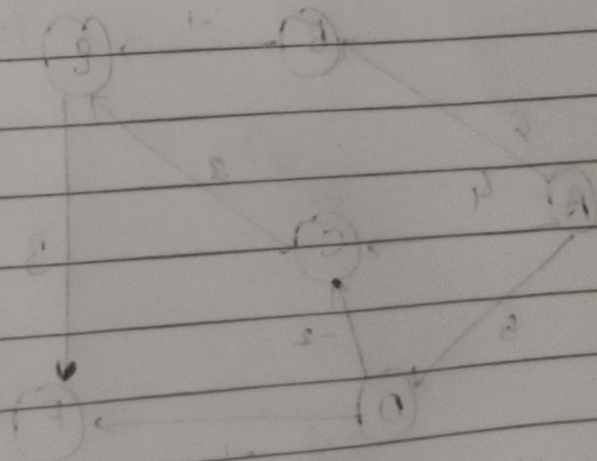
Step 5: Matrix D gives shortest distance between all pairs

• Recurrence relation

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

Where k is intermediate vertex

- Time Complexity : $O(n^3)$
- Space Complexity : $O(n^2)$



- Bellman-Ford Algorithm (SSSP)
 - Find shortest distance from a single source to all other vertices in a graph
 - Works even when edge weights are negative
 - If negative cycle exists, algorithm can detect it

Idea:

Relax all edges repeatedly

For a graph with V vertices, relax edges $V-1$ times because longest possible shortest path has $V-1$ edges

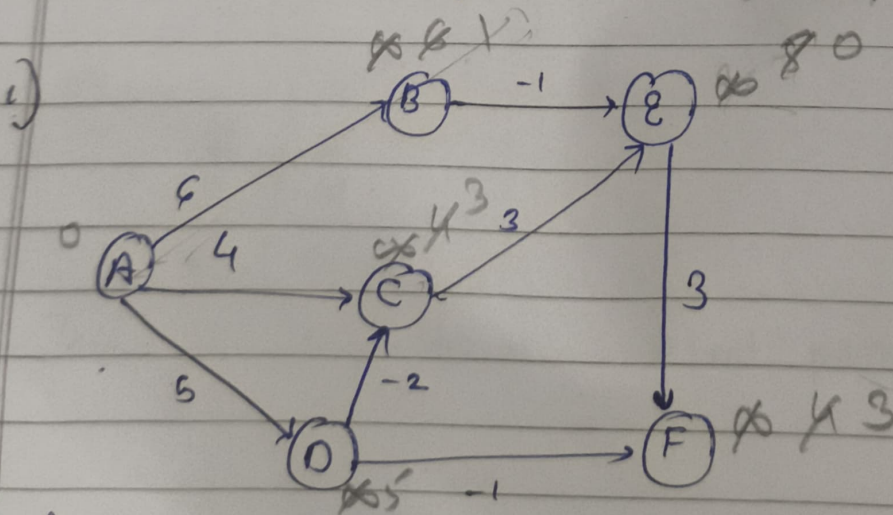
Relaxation rule:

$$\text{dist}[V] = \min(\text{dist}[U], \text{dist}[U] + w)$$

Where

$U \rightarrow V$ is an edge

w is its weight



At starting source $A=0$ and other ∞

Edges $(A,B), (A,C), (A,D), (B,E), (C,E), (D,U), (D,F), (E,F), (C,D)$

$$w = [u]b$$

1st Iteration

$$o_1 = [0]b$$

A B C D E F

1st $[u]b$ 0 4 3 5 5 4

2nd $[u]b > w + [u]b$ 3 5 0 3

3rd 0 $w + [u]b = 3[u]b$ 5 0 3

4th 0 No change

5th $[u]b$ time (u,v) spb d203 r03

$$[u]b \text{ time} > w + [u]b \text{ time}$$

$(A,B) \rightarrow (u,v)$ shortest path

$$d(B) = \min(d[B], d[A] + w)$$

$$d(B) = \min(\infty, 0 + 6) \therefore d(B) = 6$$

Vertex :

A - 0

B - 1

C - 3

D - 5

E - 0

F - 3

• Algorithm BellmanFord (G, V, E, source)

Step 1: Initialize distance array $d[]$

Step 2: For each vertex $(0, 1), (1, 2)$

$$d[v] = \infty$$

Step 3: $d[s] = 0$

Step 4: Repeat $V-1$ times

→ For each edge (u, v) with weight w

If $d[u] + w < d[v]$

$$d[v] = d[u] + w$$

Step 5: Check for negative cycle

For each edge (u, v) with weight w

If $d[u] + w < d[v]$

Graph has negative cycle

Step 6: Return $d[]$

$d = (0) \quad (2+0 \rightarrow \infty) \quad \text{ans} = (0)$

• Time complexity

$$O(V \times E)$$

• Space complexity

$$O(V)$$

$$1 \rightarrow \text{array}$$

$$0 \rightarrow A$$

$$1 \rightarrow B$$

$$2 \rightarrow C$$

$$2 \rightarrow D$$

$$0 \rightarrow E$$

$$2 \rightarrow F$$

Multistage graph :

It is a directed graph where vertices are divided into stages. Edges go only from stage $i \rightarrow$ stage $i+1$.

There is one source and one destination vertex.

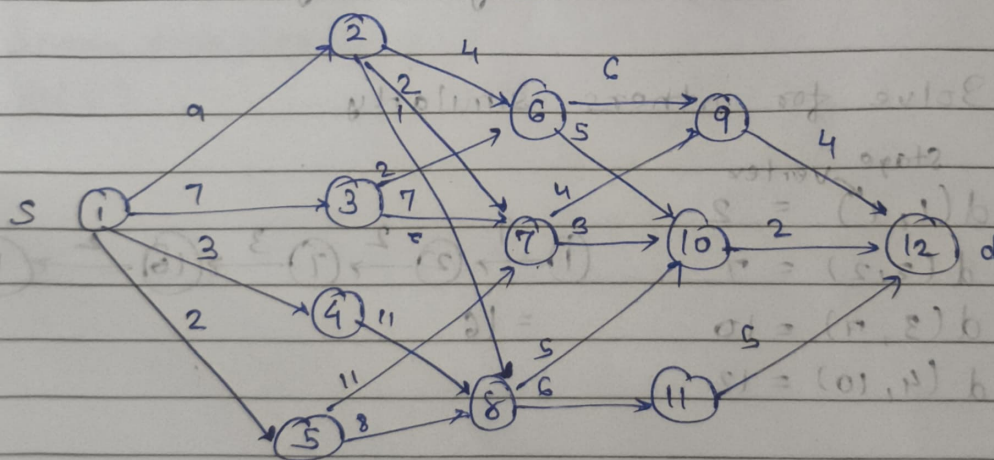
Idea : Let $cost(u) = \min \text{ cost from } u \rightarrow \text{dest}$

Formula : $cost(u) = \min (w(u, v) + cost(v))$

Formula : $cost(u) = \min (w(u, v) + cost(v))$

Formula : $cost(u) = \min (w(u, v) + cost(v))$

1) $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4$



v	1	2	3	4	5	6	7	8	9	10	11	12
cost	16	7	9	18	15	7	5	7	4	2	5	0
d	2/3	7	6	8	8	10	10	10	12	12	12	12

Start from last i.e 12

$cost(5, 12) = 0$

↑ ↑
Stage vertex

$$\text{cost}(4, 9) = 4$$

$$\text{cost}(5, 10) = 2$$

$$\text{cost}(4, 11) = 5$$

$$\begin{aligned} \text{cost}(3, 6) &= \min \left\{ \text{cost}(6, 9) + \text{cost}(4, 9) \right. \\ &\quad \left. \text{cost}(6, 10) + \text{cost}(4, 10) \right\} \\ &= \min \{ 6 + 4, \underline{5 + 2} \} \end{aligned}$$

So cost 6 = 7 given by $v = 10$

$$\begin{aligned} \text{cost}(3, 7) &= \min \left\{ \text{cost}(7, 9) + \text{cost}(4, 9) \right. \\ &\quad \left. \text{cost}(7, 10) + \text{cost}(4, 10) \right\} \\ &= \min \{ 8, \underline{5} \} \end{aligned}$$

So cost 7 = 5 and given by $v = 10$

Solve for others similarly

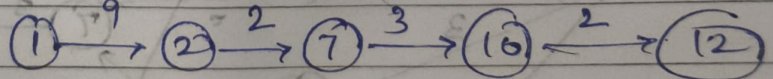
stage vertex

$$d(1, 1) = 2$$

$$d(2, 2) = 7$$

$$d(3, 7) = 10$$

$$d(4, 10) = 12$$



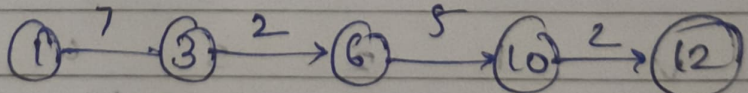
$$= 16$$

$$d(1, 1) = 3$$

$$d(2, 3) = 6$$

$$d(3, 6) = 10$$

$$d(4, 10) = 12$$



$$= 16$$

- Algorithm

Multistage Graphs (G, n)

Step 1 : Divide graphs into stages

Step 2 : let destination vertex = t

$$\text{cost}[t] = 0$$

Step 3 : For each stage from last-1 to first

For each vertex v in that stage

$$\text{cost}[v] = \min(\text{weight}(v, u) + \text{cost}[u])$$

Step 4 : Return cost (s)

- Time Complexity :

$$O(E)$$

- Space Complexity :

$$O(V)$$